Frame Invariance in FEBio Contact and Generalized-α Formulations

Conversation transcript

October 24, 2026

This transcript reproduces the user's messages and the assistant's replies. The intermediate tool work (reading files, editing code, building FEBio and running tests) is not reproduced; short notes in italics indicate where it took place.

1. Review of Section 7.6 (Tied Elastic Contact)

User: Please review section 7.6 for tied elastic contact in the attached document and let me know whether it satisfies frame invariance.

The assistant read the four pages of the attached PDF (Sections 7.5–7.7 of the FEBio Theory Manual).

Assistant:

Short answer: partly. The continuum formulation is frame-invariant. The time-discrete version in Section 7.6 is not, and the augmented-Lagrangian variant has a separate issue.

What holds up

Take a superposed rigid motion $x^* = Q\,x + c$.

- **Gap (7.6.4):** $g = x^{(2)} - x^{(1)}$. The translation c cancels, so $g^* = Q\,g$.
- **Penalty traction:** $t = \varepsilon_n\, g$, so $t^* = Q\,t$. The traction is an objective vector.
- **Surface Jacobian (7.6.3):** $J_\eta = |g_1 \times g_2|$ doesn't change under rotation.
- **Virtual work:** $\delta v^* \cdot t^* = \delta v \cdot t$, so δG_c is invariant.
- **Angular momentum:** the forces on the two sides are equal and opposite but act at points separated by g. The resulting moment is $(x^{(1)} - x^{(2)}) \times \varepsilon_n\, g = 0$ because t is parallel to g. So the pure penalty form creates no spurious moment, even with a nonzero initial gap.
- **Linearization:** (7.6.9)–(7.6.11) and the A_c (calligraphic) dual vector in (7.6.18) are consistent with these definitions. I checked the signs in (7.6.16) and (7.6.19) and they agree. The stiffness is unsymmetric because of the $D\,J_\eta$ term, which is expected.

Where it breaks

1. **Generalized-α evaluation (the main issue).** Everything is evaluated at $x_{n+\alpha_f} = \alpha_f\, x_{n+1} + (1 - \alpha_f)\, x_n$.
   - Under a time-varying rigid rotation, $x^*{n+\alpha_f} = \alpha_f\, Q{n+1}\, x_{n+1} + (1 - \alpha_f)\, Q_n\, x_n$. That is not a rotation of $x_{n+\alpha_f}$.
   - As a result, the basis vectors g_α at $n+\alpha_f$, and therefore J_η and $n_{n+\alpha_f}$, shrink under pure rigid rotation. The scale factor is $\sqrt{\alpha_f^2 + (1 - \alpha_f)^2 + 2\alpha_f(1 - \alpha_f)\cos\Delta\theta} < 1$.
   - The gap is affected the same way: a nonzero initial gap g_0 yields a traction whose magnitude changes under rigid motion alone.

- So the scheme is not incrementally objective. The error is $O(\Delta\theta^2)$ per step: negligible for small rotation increments, noticeable for large ones.

2. **Augmented Lagrangian (7.6.5).** The section doesn't say how λ is stored or updated.
   - **Objectivity:** if λ is kept as a fixed spatial vector across augmentations or time steps, it doesn't rotate with the body, so $t = \lambda + \varepsilon_n g$ is not objective. The fix is to store λ in a convected or local basis, or push it forward with the rotation.
   - **Spurious moment:** λ is generally not parallel to g, so the moment $g \times \lambda$ is nonzero until the gap converges to zero.
3. **Assumption to state explicitly.** Invariance, and the absence of a $D\eta$ term for the partner point in the linearization, both rely on $x^{(2)}$ being evaluated at fixed (tied) parametric coordinates on $\Gamma^{(2)}$.

Overall: the pure penalty formulation is objective in the continuous setting and its linearization is consistent. The generalized-α evaluation is not incrementally objective, and the augmented-Lagrangian version needs λ to be transformed with the body to be objective.

There are also a few typos: "This contact interface nearly identical" (missing "is"), "followd", and "antiymmetric".

2. A frame-invariant formulation

User: Can you generate a formulation that fixes these issues? Yes, lambda is kept a a fixed spatial vector in this implementation.

The assistant derived a corotational formulation, verified it numerically in Python (objectivity, momentum, finite-difference tangent), and wrote a revised Section 7.6 in LaTeX/PDF.

Assistant: I've rewritten Section 7.6 as a frame-invariant formulation, in LaTeX and as a compiled 4-page PDF. I checked it numerically at a single integration point. It is not yet tested in FEBio.

The three fixes:

1. **Generalized-α:** the contact forces are now computed on the real end-of-step configuration and then blended in time: f = α_f·f(x_{n+1}) + (1−α_f)·f_n, where f_n is stored from the previous step. They are no longer computed on the blended configuration x_{n+α_f}. Each force then comes from an actual configuration, so a rigid rotation can't create a false gap or area change. Every tangent term gets a factor α_f.
2. **λ storage:** λ is stored as fixed components $\hat{\lambda}$ in an orthonormal frame $R = [e_1\ e_2\ n]$ that rotates with the contact surface, and $\lambda = R\,\hat{\lambda}$. The augmentation update becomes $\hat{\lambda} \leftarrow \hat{\lambda} + \varepsilon_n R^T g$, which is unaffected by rigid motion. The linearization adds a term $D\lambda = -[\lambda]\times \Sigma\ \Omega_c\ \Delta u_c$, where Ω_c (eq. 7.6.3.4) gives how the frame rotates with nodal displacements.
3. **Angular momentum:** I added a correction term $-g\cdot\delta\lambda$ to the contact virtual work. It exactly cancels the spurious moment $g \times \lambda$. It is proportional to the gap, so it disappears at convergence and doesn't change the converged solution. With pure penalty ($\lambda = 0$), fixes 2 and 3 have no effect.

What the numerical check showed:

- **Objectivity:** under a finite rotation plus translation, the forces rotate correctly to machine precision.
- **Momentum:** net force and net moment are both zero to machine precision. Without the correction term the moment error is 0.72.
- **Tangent against finite differences:** exact when the gap is zero, with error otherwise proportional to the gap. One small derivative term is left out of the tangent. It is proportional to the gap times λ, so it can slightly slow Newton convergence before the gap closes but doesn't affect the answer.

Also worth knowing:

- **Sign error in the original:** f_b$^{(2)}$ in (7.6.15) should be negative to match the integrand $(\delta v^{(1)} - \delta v^{(2)})\cdot t$. Check whether the code already compensates for this somewhere.
- **Choice of frame:** the rotating frame follows the first surface tangent g_1. It is objective but depends on the surface parametrization. The PDF notes a polar-decomposition alternative that doesn't, which needs its own Ω_c.
- **Mixed time integration:** this assumes the solid elements may still use the blended configuration while contact uses blended forces. That combination is valid, but you should confirm it fits how your solver puts the residual together.

Files are in my working folder: tied_elastic_contact_objective.pdf, tied_elastic_contact_objective.tex, verify_tied_contact.py.

3. Fixing FETiedElasticInterface

User: The FEBio files are located here: /Users/gerard/GitHub/FEBio-safe and the specific implementation of the tied elastic interface can be found in /Users/gerard/GitHub/FEBio-safe/FEBioMech/FETiedElasticInterface. Can you please check and fix the code where needed?

The assistant read the implementation, rewrote the gap, multiplier storage, load vector and stiffness, built FEBio on Linux, and ran rotation, regression and dynamic tests.

Assistant: I've fixed `FETiedElasticInterface.cpp/.h` in FEBio-safe, and the interface is now frame-invariant. The new code compiles cleanly, but I built and tested it on Linux, not with your Xcode/MKL setup, so please rebuild and run your own models. One regression remains, described at the end. Nothing is committed.

What was actually wrong in the code (it doesn't match Section 7.6):

- **Initial gap:** the main objectivity bug isn't in the PDF. `m_Gap` (the initial gap) is stored as a fixed spatial vector, so a rigid rotation creates a false gap. In a test with an initial gap of 0.1 and a 90° rigid rotation, the top block ended up 0.14 out of place.
- **λ:** the Lagrange multiplier is a fixed spatial vector, as you said.
- **Generalized-α:** `tp.alphaf` is never used. Contact is evaluated at n+1 and the stiffness isn't scaled by α_f, so dynamic runs don't match the solid elements.

What I changed:

- **Initial gap and λ:** both are now stored in a frame that rotates with the primary surface.
- **Moment term:** a term from the rotating initial gap is added, so angular momentum is conserved exactly with penalty and up to a term proportional to the gap with augmented Lagrangian.
- **Weighting:** contact forces are now weighted by the reference area instead of the current area. This makes the forces derive from a potential, which gives an exact, symmetric stiffness matrix.
- **Generalized-α:** forces are blended as α_f·f(x_{n+1}) + (1−α_f)·f_n, with f_n stored in a new `PrepStep()`.

Test results (two non-matching hex blocks with an offset gap, built FEBio on Linux):

Test	Original	Fixed
Rotated run vs. rotated unrotated run, penalty	0.14 error	1e-12
Same, augmented Lagrangian	0.35 error	3e-4 (within the multiplier tolerance)
Newton convergence, default symmetric solver	linear	quadratic

Test	Original	Fixed
Total iterations (static, rotation + load)	146 / 659	80 / 242
Two-pass, augmented Lagrangian	0.14 error	1e-8

Behaviour changes and open issues:

- **Penalty scaling:** the penalty is now per reference area, so penalty-only results shift slightly when the surface stretches (2e-2 in a loaded test).
- **Regression:** dynamic augmented-Lagrangian runs fail when the initial gap is large (0.1, about 30% of the element size, and 0.03 combined with rotation); the original code passed both.
- **Theory PDF:** it's now out of date on three points: reference-area weighting, λ fixed within a time step, and the gap term.

4. Other contact interfaces

User: Are there similar objectivity problems with all other contact (sliding or tied) formulations? If so, can you please fix them all?

The assistant audited all contact interfaces and asked how initial gaps should be handled. The user chose "Lagged rotation" (rotate stored gaps with the surface frame once per time step).

Assistant: Yes, the same problem was in most of the tied interfaces and in the friction/stick part of four sliding ones; I've fixed all of those in FEBio-safe. Every fixed interface that I could run now gives frame-invariant results, but four of them were only compiled, not run. As you chose, the initial gap is rotated with the surface once per time step, so a small, non-accumulating error proportional to |gap| remains (7.85e-3 for a gap of 0.1, against 0.14–0.35 before).

What was wrong: these interfaces store Lagrange multiplier vectors, and some also store an initial gap, as fixed spatial vectors that never rotate with the interface.

The fix: a new shared helper, `FEBioMech/FEContactFrame.h`, defines a local frame at each contact point. At the start of every time step, each stored vector is rotated by how much that frame turned during the previous step; within a step nothing changes.

Interface	Stored vectors now rotated
tied-node-on-facet	multipliers
tied-facet-on-facet	multipliers and initial gap
sticky	multipliers
mortar-tied	nodal multipliers
tied-biphasic, tied-multiphasic	multipliers and initial gap
tied-fluid-FSI	initial gap, solid and viscous multipliers
sliding-elastic, sliding-biphasic, sliding-biphasic-mixed, sliding-multiphasic	friction/stick multiplier

Tied-elastic changed again: the initial gap now uses the same once-per-step rotation; the α_f force blending was removed because with augmented Lagrangian it made dynamic runs drift and fail; contact forces are now evaluated at the end of the step, like every other FEBio contact interface. All six dynamic augmented-Lagrangian cases pass. Reference-area weighting is kept.

Interface	Original	Fixed
tied-node-on-facet, sticky	2.8e-2	1e-11
tied-facet-on-facet	0.13	3.7e-6
tied-facet-on-facet, gap 0.1	0.27	7.85e-3
tied-elastic, gap 0	0.21	4.8e-6
tied-elastic, gap 0.1 (penalty or AL, one- or two-pass)	0.14–0.35	7.7e-3 to 7.85e-3
tied-biphasic, gap 0 / 0.1	0.21 / 0.35	3.1e-5 / 7.9e-3
sliding-elastic with friction	fails when rotated	4.4e-5
sliding-biphasic with friction	fails when rotated	4.4e-5
sliding-elastic with tension, mortar-tied	already frame-invariant	unchanged

Not run-tested: tied-multiphasic, sliding-multiphasic and tied-fluid-FSI (compiled only); sliding-biphasic-mixed failed the test model in the original code too; mortar-tied multiplier updates and tied-node-on-facet in Lagrange-multiplier mode were not reached.

Left unchanged on purpose: single normal multipliers, friction stored in surface components, rigid walls, tied-fluid. FETiedLineConstraint has the same problem but a line has no surface frame. Tied-fluid-FSI still evaluates at the generalized-α intermediate configuration.

The 25 changed files are uncommitted. The earlier standalone write-up now describes a formulation the code no longer uses.

5. Theory Manual update

User: Thanks, can you please update /Users/gerard/GitHub/FEBio-safe/Documentation/FEBio_Theory_Manua accordingly?

Assistant: I've updated the LyX file in place, but haven't opened it in LyX or rebuilt the PDF. Before saving I checked that the edited file's structure is balanced, and that all the new equations compile and every cross-reference points to an existing label.

Section 7.6, Tied Elastic Contact (rewritten):

- The contact integral is now evaluated at the end of the time step, t_{n+1}, including under generalized-α, citing Arnold and Brüls (2007), with the contraction-factor explanation.
- The traction is defined per unit reference area; the output reports the true (Cauchy) traction.
- The gap is now g − R_n·Ĝ and the multiplier λ = R_n·λ̂, with the one-step lag error of order |G|·Δθ discussed.
- A new potential-energy equation shows the contact forces are conservative, so the stiffness is symmetric.
- The linearization and discretization give the simple, symmetric penalty stiffness blocks. Every label cited elsewhere was kept.

New section, "Frame Invariance of Stored Interface Vectors", placed after Tied Fluid-FSI Interface so no section numbers change: why stored spatial vectors break frame invariance, the local surface frame, the per-step rotation update, its accuracy, and which interfaces use it.

Short pointer paragraphs were added to Tied Contact (7.5), the Sliding-Elastic and Biphasic augmented-Lagrangian schemes, and the Gap Function subsections of Tied Biphasic, Tied Multiphasic and Tied Fluid-FSI.

Arnold and Brüls (2007) is cited in plain text, not through BibTeX.

6. Merge with develop

User: I had to merge the current branch of FEBio with the develop branch. Can you please check if any of your changes were affected? If so, can you please reimplement them?

Assistant: The merge didn't affect any of my changes, so there was nothing to reimplement. All my edits are still in your working tree, uncommitted, on top of the merge commit.

- **Overlap:** five files I changed were also modified by develop; my edits are intact, with no conflict markers.
- **What develop changed there:** friction fixes, extra solute resets, and a reworked release routine — none touching the stored vectors, the start-of-step update, or the frame.
- **One behavioural change:** sliding-multiphasic now also runs its start-of-step update on a porous secondary surface even without two-pass; my multiplier rotation now applies there too, as intended.
- **Rebuild and re-test:** a full rebuild compiled with no errors and the rotation tests gave results identical to before the merge.

7. Generalized-α in the fluid-FSI solver

User: Does your presentation mean that the fluid-FSI solver also fails to satisfy objectivity because of the manner with which it implements the generalized-alpha method? If so, can you please fix the code and make changes to the LyX Theory Manual?

Assistant: Yes, the fluid-FSI solver had this problem, and in more cases than my earlier remark implied: the velocity gradient was wrong even with the default $\rho_\infty = 0$ ($\alpha_f = 1$). I've fixed it in the fluid-FSI, biphasic-FSI and multiphasic-FSI domains and updated the Theory Manual. Unlike the contact problems, this error shrinks as the time step is refined.

What was wrong:

- **Velocity gradient**, $L = (F_{n+1} - F_n)/\Delta t \cdot F_{n+\alpha_f}^{-1}$: under a pure rigid rotation its symmetric part isn't zero unless $\alpha_f = ½$. With $\alpha_f = 1$ and 0.1 rad per step, the spurious rate of deformation was 4.6% of the angular velocity.
- **Deformation gradient**, $F_{n+\alpha_f} = (1-\alpha_f)F_n + \alpha_f F_{n+1}$: spurious strain whenever $\alpha_f < 1$ (1e-3 at 0.1 rad/step, about 1e-2 at 0.3 rad/step).

The fix: the velocity gradient is evaluated on the midpoint configuration (Hughes and Winget, 1980), and stretch and rotation are interpolated separately.

Case	Original	Fixed
Fluid-FSI, $\alpha_f = 1$, 0.1 rad/step: rate of deformation / ω	4.6e-2	4e-15
Fluid-FSI, $\alpha_f = 0.667$, 0.1 rad/step: rate of deformation / ω	1.6e-2	4e-15
Fluid-FSI, $\alpha_f = 0.667$, 0.1 rad/step: strain	1.0e-3	8e-16
Fluid-FSI, $\alpha_f = 0.5$, 0.3 rad/step: strain	1.0e-2	5e-16
Biphasic-FSI, $\alpha_f = 1$ and 0.667	same spurious values	zero to machine precision

At the time, the tangent stiffness had not been updated (iterations rose from 60 to 94 at $\alpha_f < 1$),

and the same pattern was noted in the solid and shell domains. The Theory Manual's FSI Time Integration section was updated accordingly.

8. Extra-rotation tangent terms and solid domains

User: Please add the extra-rotation derivative terms. Please fix the solid domain formulations as well and update the manual accordingly.

The assistant derived the exact derivative of the objective interpolation, moved the kinematics to `FECore/FEObjectiveKinematics.h`, made the intermediate-configuration geometry objective in `FESolidDomain` and `FESSIShellDomain`, rewrote the displacement tangents, and verified them with FEBio's `stiffness_test` diagnostic.

Assistant: The extra-rotation terms are in, and the solid, shell and FSI domains now have exact tangents for the objective interpolation, except the EAS and ANS shells. FEBio's finite-difference stiffness check agrees to about 1e-10 relative error at α_f = 1 and 0.667 for every domain listed below. Everything is built and tested on Linux only and uncommitted.

I broke something last turn, now fixed. The biphasic-FSI and multiphasic-FSI tangents were already exact (~1e-11) in your code. My midpoint velocity gradient from last turn made them inconsistent again: 7% error even at α_f = 1. They're exact again now.

The fluid-FSI tangent was already off before any of my changes: 20–50% errors in the w–u and ef–u blocks, even at α_f = 1. It used the Newmark factor γ/(βΔt) for the rate derivatives, while the residual uses finite differences. I replaced its displacement columns with a new `ElementStiffnessDisplacement`. Total iterations on the shear test at ρ∞ = 0.5 are back to 60.

What changed:

- **`FECore/FEObjectiveKinematics.h`** (new): `FEObjectiveInterpolation`, with the exact derivative of the interpolated F with respect to F_{n+1} (checked against finite differences to 1e-10), and the derivative of the midpoint velocity gradient.
- **Intermediate-configuration geometry:** for α_f < 1, the Jacobian, determinant and base-vector functions in `FESolidDomain` and `FESSIShellDomain` now use the objective F_{n+α_f}.
- **Exact stiffness for α_f < 1** in the standard solid, 3-field solid, elastic shell and 3-field shell domains. The α_f = 1 code paths are unchanged, and static results match the original code to 1e-14.
- **Bug fixes along the way:** the fluid-FSI body-force stiffness was writing Kwu entries into the ww block; `tens4dmm::dot` drops one of the two shear terms, so the full tensor is contracted explicitly.

Domain	Tangent error at α_f < 1, before → after	Spurious strain under rigid rotation, before → after
Fluid-, biphasic- and multiphasic-FSI	(see above) → ~1e-11	1e-3 → 1e-15
Standard solid	already exact → 7e-10	4e-3 → 7e-16
3-field solid	75% → 7e-10	4e-3 → 7e-16
Elastic shell	96% → 1e-9	4e-3 → 9e-16
3-field shell	55% → 3e-10	4e-3 → 9e-16

At ρ∞ = 0.5, iterations dropped from 112 to 30 (3-field solid), 61 to 33 (shell) and 132 to 40 (3-field shell).

Not done or still open:

- **EAS and ANS shells:** kinematics now objective, but their tangents were already 66% off at α_f < 1 before these changes.

- **Other elements:** UT4, UDG, SRI and trusses never used the intermediate configuration.
- **Symmetric solver:** the exact tangent at $\alpha_f < 1$ is not symmetric, so it needs the non-symmetric solver.
- **Already inexact before:** viscous-solid material tangents (leading order only) and the $\alpha_f = ½$ energy-conserving stress correction (not linearized).
- **Small gaps:** biphasic/multiphasic-FSI body-force stiffness still uses the old geometric term; in the multiphasic-FSI check the analytic solute–solute block is zero in both versions.
- **Behaviour change:** the midpoint velocity gradient also applies to quasi-static analyses, so viscous-solid and biphasic results shift by $O(\Delta t)$.

Theory Manual (LyX): the elastodynamics generalized-α section (objective F_{n+α_f}, mid-point velocity gradient, updated linearizations, a new equation for the derivative of the interpolation, the non-dyadic stiffness column) and the FSI Time Integration section (exact displacement tangent) were updated. All new equations compile with pdflatex.

`FECore/FEObjectiveKinematics.h` is a new file, so it needs a `git add`.

9. This document

User: Please generate a PDF of this entire exchange.